

浙江大学 实验报告

姓名：马玉玺

学号：3240104676

日期：2025 年 8 月 23 日

实验名称：向量化进阶(RISC-V)

一、任务一优化思路

1. 向量化最内层循环

矩阵乘法中计算量最大的部分是执行点积的最内层循环（即 `kk` 循环）。优化的核心思想就是用功能强大的向量指令来替代这个一次只处理一对元素的标量循环，从而实现一次处理多个元素。

优化前：

```
// 每次循环只进行一次乘法和一次加法
for (int kk = 0; kk < k * 4; kk++) {
    C[i*n+j] += A[i*k*4+kk] * B[j*k*4+kk];
}
```

优化后（向量化实现）：

`__riscv_vle8_v...`：不再是从 A 矩阵和 B 矩阵中一次取一个 `uint8_t` 和一个 `int8_t`，而是一条指令就从两个矩阵中加载一大块元素到宽大的向量寄存器中（`va` 和 `vb`）。

`__riscv_vwmulsu_vv_i16m2`：一条指令就能完成向量 `va` 中的所有元素与 `vb` 中对应元素的相乘。

`__riscv_vwadd_wv_i32m4`：一条指令就能将所有乘法结果累加到总和向量（`v_sum`）中。

通过这种方式，CPU 执行更少的指令就能完成同样多的计算，从而极大地提升了运算速度。

2. 动态向量长度（`vsetvl`）

RISC-V 向量扩展的一个强大之处在于，我们无需在编码时知道硬件向量寄存器的确切长度（例如是 128 位还是 256 位）。

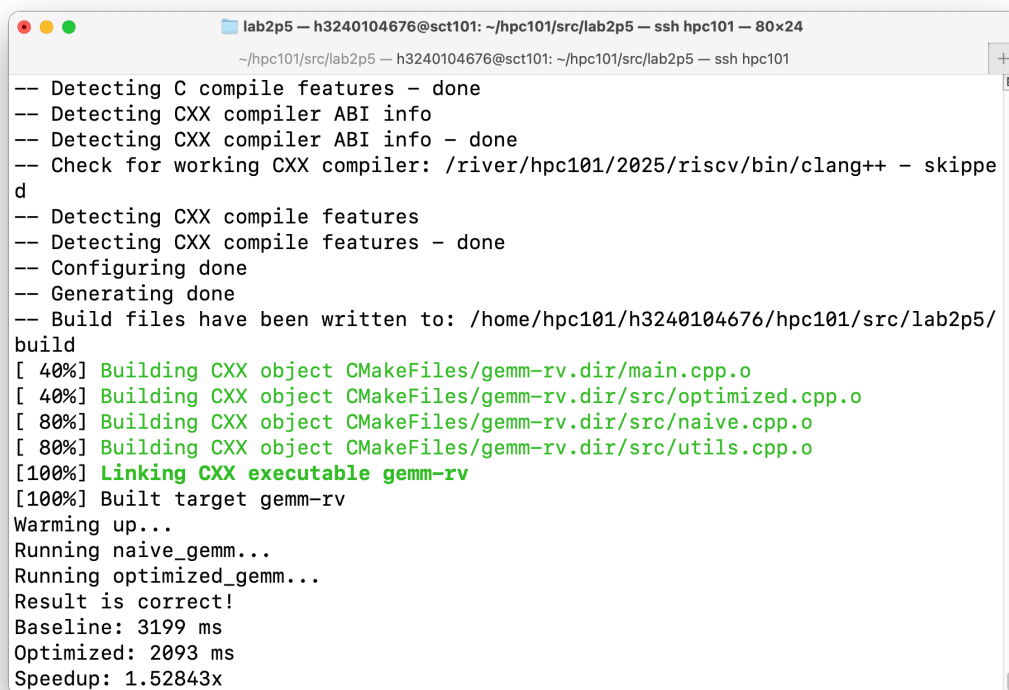
`__riscv_vsetvl_e8m1(k * 4 - kk)` 这条指令就是实现这一点的“魔法”。在每次执行向量操作前，它会询问硬件：“我剩下 `k * 4 - kk` 个元素需要处理，你一次能处理多少个？”硬件会根据自身能力，将向量长度（`vl`）设置为当前迭代能处理的最大值。这意味着同一份编译好的代码，可以在不同向量计算能力的机器上高效运行。更重要的是，这个机制自动处理了循环的“尾巴”。如果循环末尾只剩下 5 个元素，`vl` 会被设置为 5，向量指令将只处理这 5 个元素，从而无需编写额外的标量代码来处理余数。

4. 向量规约

在 `kk` 循环结束后，向量寄存器 `v_sum` 中存放着多个部分和。例如，如果硬件一次处理 16 个元素，`v_sum` 中就会有 16 个独立的累加值。我们需要将它们全部加起来，才能得到 `C[i*n+j]` 的最终结果。

`__riscv_vredsum_vs_i32m4_i32m1` 这条指令执行的就是规约求和 (reduction sum) 操作。它接收一个向量 `v_sum`，将其内部的所有元素相加，最终产生一个只包含单个结果的向量 (`v_scalar_sum`)。最后，`__riscv_vmv_x_s_i32m1_i32` 将这个标量结果从向量计算单元移回一个标准的 CPU 寄存器，以便存入 C 矩阵中。

总结来说，这段代码巧妙地利用了加宽操作来保证计算的正确性，并利用动态向量长度机制来确保代码的可移植性和简洁性。



```
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /river/hpc101/2025/riscv/bin/clang++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/hpc101/h3240104676/hpc101/src/lab2p5/build
[ 40%] Building CXX object CMakeFiles/gemm-rv.dir/main.cpp.o
[ 40%] Building CXX object CMakeFiles/gemm-rv.dir/src/optimized.cpp.o
[ 80%] Building CXX object CMakeFiles/gemm-rv.dir/src/naive.cpp.o
[ 80%] Building CXX object CMakeFiles/gemm-rv.dir/src/utils.cpp.o
[100%] Linking CXX executable gemm-rv
[100%] Built target gemm-rv
Warming up...
Running naive_gemm...
Running optimized_gemm...
Result is correct!
Baseline: 3199 ms
Optimized: 2093 ms
Speedup: 1.52843x
```

Figure 1: 任务一运行结果

```
lab2p5 — gaia@MacBook-Air — zsh — 80x24
..01/src/lab2p5

[100%] Linking CXX executable gemm-rv
[100%] Built target gemm-rv
exit code: 0
2025-08-25 12:44:51.711 running workflow 1 step 3 / 3
2025-08-25 12:44:51.807 running workflow 2 / 3
2025-08-25 12:44:51.973 running workflow 2 step 1 / 2
2025-08-25 12:44:55.059 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -c 8 -p
riscv taskset 1 ./gemm-rv"
Warming up...
Optimized: 2093 ms
INT8: 0.0046968 TOPS / core (Utilization: 1.1742%)
result is correct
exit code: 0
2025-08-25 12:44:57.535 running workflow 3 / 3
2025-08-25 12:44:57.753 running workflow 3 step 1 / 1
2025-08-25 12:45:01.461 Submission completed
Submit is completed
Message:
    judge successfully finished
Score 50.75 max.100 (Unweighted)
Judgement Message:
    Correct,time: 2093ms
```

Figure 2: 任务一 OJ 运行结果

二、任务二优化思路

核心优化思路是 Loop Unrolling。

在未优化的版本中，最内层的 `kk` 循环是性能热点。在这个循环里，程序重复地执行“`memcpy` -> `vle` -> `vmadotus`”这个流程。虽然 `vmadotus` 本身很快，但循环本身带来的开销（如 `kk` 的增加、循环条件的判断和跳转）累积起来会严重影响整体性能。

优化前（一次处理一个数据块）：

```
for (kk = 0; kk < k*4; kk += K_TILE) {
    // 准备并处理一个 4x8 的数据块...
}
```

这份代码的核心改动是将 `kk` 循环展开了 2 次。

优化后（一次处理两个数据块）：

```
for (kk = 0; kk < k*4 - K_TILE; kk += 2 * K_TILE) {
    // --- 第 1 块 ---
    // 准备数据 (buffer0)
    // 计算 (va0, vb0)

    // --- 第 2 块 ---
    // 准备数据 (buffer1)
```

```

    // 计算 (va1, vb1)
}

```

展开循环主要带来了两大好处：

减少循环开销：通过一次循环执行两次迭代的工作量，循环控制指令（`kk` 的增加、条件判断、跳转）的总执行次数几乎减半了。对于一个执行成千上万次的紧凑循环来说，这能节省大量 CPU 时间。

提升指令级并行度 (ILP): CPU 可以同时执行多条互不依赖的指令。循环展开为 CPU 提供了一个更大、更丰富的指令窗口。

在展开的循环体内部，准备第一个数据块（`a_buffer0`, `b_buffer0`）的 `memcpy` 操作和准备第二个数据块（`a_buffer1`, `b_buffer1`）的操作是完全独立的。CPU 可以并行地处理这些内存拷贝，或者在等待第一次计算（`vmadotus(..., va0, vb0)`）的结果时，提前开始执行第二次的数据加载（`vle8_v_u8m1(a_buffer1, ...)`），这就是所谓的软件流水线思想。它有效地隐藏了内存访问的延迟，让计算单元和内存单元尽可能地同时保持“繁忙”状态，从而提升了硬件的利用率。

代码说明：

1. 主循环：一个新的 `for` 循环（`kk += 2 * K_TILE`）负责处理大部分工作，它一次处理两个 `4x8` 的数据块。
2. 收尾循环：在主循环结束后，增加了一个传统的 `for` 循环（`kk += K_TILE`）。它的作用是处理“剩余”部分。例如，如果总共有奇数个数据块，那么最后一个无法配对的数据块就会由这个收尾循环来处理，保证了算法的正确性。

```
lab2p5 — h3240104676@sct101: ~/hpc101/src/lab2p5 — ssh hpc101 — 80x24
~/hpc101/src/lab2p5 — h3240104676@sct101: ~/hpc101/src/lab2p5 — ssh hpc101

CPU load: 1.65 2.60 2.70
Memory: 62118M (24%) out of 251.7G
Swap: not available
See https://go.zjusct.io/opendocs for more information.
See https://clusters.zju.edu.cn for cluster metrics.
Last login: Mon Aug 25 20:49:35 2025 from 172.25.4.253
h3240104676@sct101:~$ cd hpc101/src/lab2p5
h3240104676@sct101:~/hpc101/src/lab2p5$ ./run.sh
-- Configuring done
-- Generating done
-- Build files have been written to: /home/hpc101/h3240104676/hpc101/src/lab2p5/
build
[ 20%] Building CXX object CMakeFiles/gemm-rv.dir/src/optimized.cpp.o
[ 40%] Linking CXX executable gemm-rv
[100%] Built target gemm-rv
Warming up...
Running naive_gemm...
Running optimized_gemm...
Result is correct!
Baseline: 3201 ms
Optimized: 2094 ms
Speedup: 1.52865x
h3240104676@sct101:~/hpc101/src/lab2p5$
```

Figure 3: 任务二运行结果

```
lab2p5 — gaia@MacBook-Air — -zsh — -zsh — 80x24
..01/src/lab2p5

[ 80%] Building CXX object CMakeFiles/gemm-rv.dir/src/utils.cpp.o
[100%] Linking CXX executable gemm-rv
[100%] Built target gemm-rv
exit code: 0
2025-08-25 12:36:44.360 running workflow 1 step 3 / 3
2025-08-25 12:36:44.470 running workflow 2 / 3
2025-08-25 12:36:44.649 running workflow 2 step 1 / 2
2025-08-25 12:36:47.743 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -c 8 -p
riscv taskset 1 ./gemm-rv"
Warming up...
Optimized: 2711 ms
INT8: 0.00362612 TOPS / core (Utilization: 0.906529%)
result is correct
exit code: 0
2025-08-25 12:36:50.872 running workflow 3 / 3
2025-08-25 12:36:51.087 running workflow 3 step 1 / 1
2025-08-25 12:36:54.791 Submission completed
Submit is completed
Message:
judge successfully finished
Score 24.94 max.100 (Unweighted)
Judgement Message:
Correct,time: 2711ms
```

Figure 4: 任务二 OJ 运行结果